

Lab 12 — ¿Por qué 8 núcleos no son 8× más rápido?

Computación Paralela y Distribuida (CC3069) · Sep 30, 2026 · @Luis

Objetivo

Vas a medir cuánto más rápido corre un programa al agregar hilos y vas a descubrir, con tus propios datos, por qué casi nunca llega a ser p veces más rápido. Ya viste en clase speedup, eficiencia, las cuatro fugas, la suma en dos fases y la efectividad (hasta la diapositiva 18): aquí los pones a prueba en tu máquina. Lo que sigue —fracción paralelizable, Amdahl, Gustafson, speedup superlineal y cómo medir bien— lo vas a descubrir con tus datos antes de verlo en clase.

Al terminar podrás:

- Medir tiempos de ejecución de forma confiable con `omp_get_wtime()`.
- Calcular speedup, eficiencia, costo y efectividad a partir de mediciones reales.
- Identificar qué limita el speedup: la parte secuencial, el overhead, el desbalance de carga.
- Distinguir entre escalar con un problema fijo y escalar con un problema que crece.

Modalidad: parejas · **Duración estimada:** 2 horas · **Lenguaje:** C con OpenMP · **Entregable:** reporte PDF + CSVs + gráficas (ver al final).

Regla del lab: antes de cada experimento escribe tu **predicción** en el reporte. Se califica que la hayas hecho y que expliques la diferencia con lo medido, no que acertaras.

Preparación (15 min)

Descarga `lab12_starter.zip` del portal. Contiene `lab12_speedup.c` (los 4 experimentos), un `Makefile` y `graficar.py` (calcula S, E, C y F y genera las gráficas).

Sistema	Instalar	Compilar
Linux / WSL	<code>sudo apt install build-essential</code> <code>python3-matplotlib</code>	<code>make</code>

Sistema	Instalar	Compilar
macOS (clang)	<code>brew install libomp</code> y <code>pip3 install matplotlib</code>	<code>make</code>
macOS (gcc)	<code>brew install gcc</code>	<code>make CC=gcc-14</code> (o la versión que instaló)
Windows sin WSL	Instala WSL: <code>wsl --install</code>	<code>make</code> dentro de WSL

Verifica que funciona con `./lab12` (sin argumentos debe mostrar el uso).

Anota en tu reporte los datos de tu equipo — sin esto tus resultados no se pueden interpretar:

1. Modelo de CPU y cuántos núcleos físicos y lógicos tiene (`lscpu` en Linux/WSL, `sysctl -n machdep.cpu.brand_string hw.physicalcpu hw.logicalcpu` en macOS).
2. Si tu CPU tiene núcleos de rendimiento y de eficiencia (Apple M-series, Intel 12.^a gen en adelante), cuántos de cada uno.
3. Si tiene Hyper-Threading / SMT.

Para medir bien: laptop conectada a la corriente, navegador y apps cerradas, y no uses la computadora mientras corre un experimento. El programa repite cada medición 5 veces y reporta la mediana.

Repaso: lo que ya viste en clase

Usamos la misma notación de clase. **T_s** = tiempo de la versión 100% secuencial (sin OpenMP); **T_p** = tiempo con p hilos.

Medida	Fórmula	Qué responde
Speedup	$S = T_s / T_p$	¿cuántas veces más rápido?
Eficiencia	$E = S / p$	¿qué fracción de cada hilo se aprovechó?
Costo	$C = p \cdot T_p$	¿cuántos hilo-segundos pagué?
Efectividad	$F = S / C$	¿cuánta rapidez obtengo por unidad de costo?

Y las cuatro fugas de eficiencia: η_f parte secuencial, η_c comunicación y sincronización, η_b desbalance, η_r trabajo redundante. En cada parte vas a identificar cuál estás viendo.

El programa mide; tú calculas. `graficar.py` también calcula S, E, C y F para que verifiques: en el reporte muestra el cálculo a mano de al menos dos filas.

Cada experimento imprime un CSV con estas columnas:

Columna	Significado
<code>p</code>	número de hilos usados
<code>t_seq</code>	Ts: versión sin OpenMP del mismo trabajo (s)
<code>t_par</code>	Tp: versión OpenMP con p hilos (s)
<code>t_parte_seq</code>	dentro de Tp, lo que tardó la parte que no se puede repartir (s)
<code>param</code>	el parámetro del experimento (fracción paralelizable fp, schedule, etc.)

Parte 1 — La parte que no se puede repartir (35 min)

El modo `amdahl` ejecuta W unidades de trabajo. Una fracción **fp** (fracción paralelizable) se reparte entre p hilos con `#pragma omp parallel for`; el resto, **1 - fp**, es una cadena donde cada paso depende del anterior. Es la fuga η_f en estado puro. El tamaño del problema **no cambia** al agregar hilos.

Predicción (antes de correr): con tu número máximo de hilos, ¿qué speedup esperas para fp = 1.00? ¿y para fp = 0.90? Escríbelo.

```
./lab12 amdahl 400000 1.00 > amdahl_fp100.csv
./lab12 amdahl 400000 0.95 > amdahl_fp95.csv
./lab12 amdahl 400000 0.90 > amdahl_fp90.csv
./lab12 amdahl 400000 0.75 > amdahl_fp75.csv
./lab12 amdahl 400000 0.50 > amdahl_fp50.csv
python3 graficar.py amdahl_*.csv
```

Llena esta tabla en tu reporte (pmax = tu número de hilos):

fp	S (p=2)	S (p=4)	S (pmax)	E (pmax)	p con F máxima
1.00					
0.95					
0.90					
0.75					
0.50					

Preguntas

1. Con $fp = 1.00$ ¿llegaste a $S = p$? Si no, propón al menos dos causas pensando en tu hardware (núcleos de eficiencia, Hyper-Threading, turbo boost cuando corre un solo hilo, costo de crear hilos). ¿Alguna encaja en las cuatro fugas?
2. Describe la forma de la curva de S cuando $fp = 0.75$. ¿Qué pasa con la ganancia de agregar el hilo número 8 comparada con la del hilo 2?
3. **Deriva tu propio modelo.** Un programa tarda T_1 segundos en un núcleo; una fracción fp de ese tiempo se puede repartir entre p hilos y el resto no. Escribe T_p en función de T_1 , fp y p . Luego escribe $S(p) = T_1 / T_p$ y simplifica para que T_1 desaparezca.
4. Usa tu fórmula para predecir $S(p_{max})$ con $fp = 0.90$ y compárala con lo medido. ¿Qué tan cerca quedó?
5. ¿A qué valor tiende tu fórmula cuando $p \rightarrow \infty$? Con $fp = 0.90$, ¿cuál es el speedup máximo posible aunque tuvieras 1000 núcleos?
6. **Al revés:** despeja fp de tu fórmula y calcúlalo con el S que mediste para $fp = 0.90$ y $p = p_{max}$. ¿Te da menos de 0.90? ¿Qué fuga representa esa diferencia?
7. **Efectividad:** para $fp = 0.90$, ¿con cuántos hilos es máxima F ? ¿Es el mismo p que da el menor tiempo? Compáralo con el caso de 16 números de clase, donde el óptimo fue $p = 4$.
8. Un programa tarda 20 s y 2 s de ellos son leer un archivo de forma secuencial. Tu jefe quiere comprar un servidor de 64 núcleos. ¿Qué le respondes, con números de speedup y de efectividad?

Parte 2 — ¿Vale la pena paralelizar algo pequeño? (25 min)

2a. El Ts honesto. Vuelve a `amdahl_fp100.csv` y compara `t_seq` con `t_par` en la fila $p =$

1. Son el mismo trabajo: uno sin OpenMP y otro con OpenMP usando un solo hilo.

1. ¿Cuál es más rápido y por cuánto (en %)?

2. Si alguien reporta su speedup usando como T_s el programa paralelo con 1 hilo, ¿su speedup sale inflado o desinflado? ¿Por qué es tramposo?

2b. Sumar N números. El modo `suma` suma N números con un `reduction(+:s)`, para N desde 16 hasta 16.7 millones. Para que los tiempos sean medibles, cada suma se repite muchas veces y se reporta el tiempo de **una** suma.

Predicción: ¿a partir de qué N crees que la versión paralela empieza a ganarle a la secuencial?

```
./lab12 suma > suma.csv
python3 graficar.py suma.csv
```

Preguntas

3. Con $N = 16$ y $p = p_{\max}$, ¿cuánto vale S ? Exprésalo como "la versión paralela es X veces más lenta".
4. En clase, con el modelo de dos fases, 16 números y $p = 4$ daban $T_p = (16/4 - 1) + \log_2 4 = 5$ sumas y $S = 3$. ¿Qué supuesto de ese modelo no se cumple en tu máquina? ¿Qué fuga es?
5. Lista lo que OpenMP tiene que hacer en cada `parallel for` además de sumar (despertar hilos, repartir, combinar el `reduction`, esperar en la barrera...). ¿En qué se va el tiempo con $N = 16$?
6. Estima a partir de qué N la versión paralela empieza a convenir en tu equipo. ¿Qué regla práctica sacas sobre cuándo vale la pena un `parallel for`?
7. Con N grande, ¿llega S a p ? La suma hace muy poco cálculo por cada dato que lee de memoria; ¿qué recurso compartido entre todos los núcleos podría estar limitando? ¿Encaja en alguna de las cuatro fugas?

Parte 3 — Cuando a un hilo le toca más trabajo (20 min)

El modo `desbalance` tiene un ciclo donde la iteración i cuesta proporcional a i : las primeras son baratas y las últimas caras. Es la fuga η_b (desbalance) de la clase, ahora medida. El trabajo total es el mismo; lo que cambia es **cómo se reparten** las iteraciones (`schedule`).

- `static`: se parte el rango en p bloques contiguos iguales antes de empezar.
- `dynamic`: cada hilo toma bloques de 64 iteraciones a medida que se desocupa.
- `guided`: como `dynamic`, pero los bloques empiezan grandes y se van achicando.

Predicción: con $p = 4$ y `static`, ¿qué fracción del trabajo total le toca al último hilo? ¿Qué speedup esperas entonces?

```
./lab12 desbalance static > desb_static.csv
./lab12 desbalance dynamic > desb_dynamic.csv
./lab12 desbalance guided > desb_guided.csv
python3 graficar.py desb_*.csv
```

Preguntas

1. Compara $S(p_{\max})$ de los tres schedules. ¿Cuál gana y por cuánto?
2. Con `static`, calcula a mano qué fracción del trabajo le toca al hilo más cargado para $p = 2$ y $p = 4$ (pista: el costo crece como una recta, el trabajo de un bloque es el área de un trapecio). Con eso predice S y compáralo con lo medido.
3. En `static`, ¿qué están haciendo los primeros hilos mientras el último termina? ¿Cómo se ve eso en la eficiencia?
4. Si `dynamic` es mejor aquí, ¿por qué no usarlo siempre? Piensa qué costo extra tiene repartir en tiempo de ejecución y relacionalo con lo que viste en la Parte 2.
5. Si tu CPU tiene núcleos de rendimiento y de eficiencia, ¿por qué un `static` con iteraciones de costo idéntico también podría quedar desbalanceado?

Parte 4 — ¿Y si el problema crece con los núcleos? (25 min)

En la Parte 1 el problema era fijo. Ahora el modo `gustafson` mantiene fija la parte secuencial (20 000 unidades) pero le da a **cada hilo** 100 000 unidades paralelas: con p hilos el problema total es $20\,000 + 100\,000 \cdot p$. `t_seq` es lo que tardaría ese problema más grande en un solo núcleo.

Predicción: con $p = 1$ la parte secuencial es $1/6$ del trabajo ($\sim 17\%$). Según tu fórmula de la Parte 1, ¿qué speedup máximo tendría un programa con $f_p = 0.83$? ¿Crees que aquí pase lo mismo?

```
./lab12 gustafson > gustafson.csv
python3 graficar.py gustafson.csv
```

Preguntas

1. ¿Cómo cambian `t_par` y `t_seq` conforme crece p ? Explica por qué uno casi no se mueve y el otro crece.

2. Compara $S(p_{\max})$ con lo que predice tu fórmula de la Parte 1 para $f_p = 0.83$. ¿Por qué aquí el resultado es tan distinto si el código es el mismo?
3. Calcula $s = t_{\text{parte_seq}} / t_{\text{par}}$ para cada fila. ¿Se mantiene o cambia con p ?
4. **Deriva el segundo modelo.** Si en la ejecución paralela una fracción s de T_p es secuencial y el resto $(1 - s) \cdot T_p$ lo hacen p hilos en paralelo, ¿cuánto tardaría ese mismo trabajo en un solo núcleo? Escribe $S = T_s / T_p$ en función de s y p . Verifica tu fórmula con tus datos.
5. Cada experimento responde una pregunta distinta: "¿cuánto más rápido resuelvo el mismo problema?" y "¿qué tanto más grande puedo hacer el problema en el mismo tiempo?". Asocia cada pregunta con la Parte 1 o la Parte 4 y da un ejemplo real de cada una (pronóstico del clima, renderizado, entrenamiento de modelos, una consulta web...).

Síntesis, reto y entrega

Síntesis (obligatoria). Para cada parte, di qué fuga (η_f , η_c , η_b o η_r) explica la eficiencia que perdiste, con el número medido que lo demuestra. Luego lista lo que **no** encaja en ninguna de las cuatro (pistas: preguntas 1.1 y 2.7). Tráelo a la clase: es justo lo que vemos después de la efectividad.

Reto opcional (+10). Consigue un speedup **mayor que p** ($S > p$) y explica por qué ocurre, o demuestra que el que obtuviste era falso.

- Pista 1: cada núcleo tiene su propia caché. ¿Qué pasa si el arreglo completo no cabe en la caché de un núcleo, pero cada pedazo sí cabe en la caché de cada núcleo? Puedes cambiar los tamaños de `tamanos[]` en `exp_suma`.
- Pista 2: un $S > p$ que desaparece al repetir la medición no cuenta. Reporta al menos 5 corridas.

Entrega: un ZIP con el reporte en PDF, todos los CSV y las gráficas PNG. El reporte incluye los datos de tu equipo, tus predicciones, las tablas y las respuestas numeradas como en esta guía.

Criterio	Puntos
Datos del equipo, CSVs y gráficas completos	15
Predicciones escritas y contrastadas con lo medido	15

Criterio	Puntos
Parte 1 — tabla, modelo derivado y preguntas	25
Parte 2 — Ts honesto y granularidad	15
Parte 3 — desbalance de carga	15
Parte 4 — problema que crece y segundo modelo	15
Reto opcional	+10